ECC-Integrated Secure Boot and ASCON-Based Authenticated Communication for Embedded IoT Devices

Rishav Kumar, Jayaditya Dutta, Ayush Pathak, Sneha Selot, Albin Thomas Jiji, Arya Mishra
School of Electronics Engineering, VIT Chennai, Chennai, India
Faculty Mentor: Dr. Balakrishnan R

Abstract

The proliferation of resource-constrained Internet of Things (IoT) devices has intensified the need for lightweight, hardware-friendly security mechanisms that protect both firmware integrity and communication confidentiality. This paper presents the design and Raspberry Pi-based implementation of a layered security pipeline for embedded IoT nodes, combining Elliptic Curve Cryptography (ECC) based secure boot verification with an ASCON-128a authenticated encryption channel. The system integrates four cryptographic stages: (i) ECDSA-based secure boot verification of firmware integrity over the NIST P-256 curve, (ii) an Elliptic Curve Diffie-Hellman (ECDH) key exchange for establishing a shared secret between communicating nodes, (iii) HKDF-SHA256-based derivation of session keys, and (iv) mutual ECDSA challenge-response authentication followed by ASCON-128a authenticated encryption with associated data (AEAD) for payload confidentiality and integrity, concluding with a defined secure-shutdown procedure. Unlike the project's original design, which specified AES-256-GCM as the payload cipher, the implemented and validated system replaces AES-256-GCM with ASCON-128a, a permutation-based lightweight AEAD cipher better suited to constrained embedded targets. The complete pipeline has been implemented and functionally validated in Python on a Raspberry Pi platform; a native C/mbedTLS port of the ASCON-128a stage to the ESP32 microcontroller is identified as future work and has not yet been implemented. Experimental validation demonstrates correct secure boot verification and tamper rejection, matching ECDH shared secrets and HKDF-derived keys between communicating parties, successful mutual ECDSA authentication, and correct ASCON-128a encryption, decryption, and tamper detection, including rejection of a single-bit ciphertext modification. The results indicate that the proposed pipeline offers a practical, lightweight alternative to AES-GCM for secure boot and authenticated communication in embedded IoT deployments.

Keywords—*secure boot; elliptic curve cryptography; ECDH; ECDSA; HKDF; ASCON-128a; authenticated encryption; embedded IoT security; Raspberry Pi; lightweight cryptography*

1. INTRODUCTION AND PROBLEM STATEMENT

The rapid expansion of Internet of Things (IoT) deployments across industrial, medical, and consumer domains has exposed embedded devices to an increasingly hostile threat landscape. Many of these devices execute safety- or privacy-critical functions while operating under tight constraints on processing power, memory, and energy, making conventional, computationally heavy cryptographic stacks difficult to justify. Two threats are of particular concern for such deployments: (i) unauthorized modification of device firmware, which can be used to plant persistent malware or disable safety functions, and (ii) interception or tampering of data exchanged between devices, which can compromise the confidentiality and integrity of sensitive telemetry such as patient health records or industrial sensor readings.

A robust embedded security architecture must therefore address firmware trust and communication security jointly, rather than in isolation. Secure boot mechanisms, typically built on digital signature verification, establish that a device is executing only firmware that has been authorized by a trusted issuer. Once a device is known to be running trusted code, a second layer is required to protect subsequent communication: a key-establishment protocol to derive a shared secret, a mutual authentication step to confirm the identity of the communicating peer, and an authenticated encryption scheme to protect the confidentiality and integrity of exchanged data.

This work targets that combined problem for embedded IoT nodes represented by the Raspberry Pi and, in a future extension, the ESP32 microcontroller. The original design of this project specified AES-256-GCM as the payload authenticated-encryption cipher. However, AES-256-GCM's block-cipher structure and its reliance on hardware AES acceleration for efficient operation make it a comparatively heavyweight choice for the most constrained class of microcontrollers targeted by this project. This paper reports a revised design and Raspberry Pi implementation in which the payload cipher is replaced with ASCON-128a, a permutation-based, sponge-construction AEAD cipher selected by the National Institute of Standards and Technology (NIST) as the basis of its lightweight cryptography standard. The remainder of the paper documents the resulting five-stage pipeline—secure boot, ECDH key exchange, HKDF key derivation, mutual ECDSA authentication, and ASCON-128a authenticated encryption—along with its Raspberry Pi implementation and experimental validation.

The specific contributions of this paper are: (1) a secure boot and authenticated-communication architecture for embedded IoT nodes that combines ECC-based asymmetric primitives with a lightweight symmetric AEAD cipher; (2) a working Python

implementation of the full pipeline on the Raspberry Pi, validated through functional tests covering both the correct-operation and adversarial (tamper) cases; and (3) an explicit account of the current implementation boundary, clarifying that the ASCON-128a stage has been implemented and validated on the Raspberry Pi only, with ESP32 support identified as future work requiring a native C implementation using mbedTLS.

2. RELATED WORK AND RESEARCH GAP

Secure boot mechanisms based on asymmetric digital signatures are well established in the embedded systems literature and in industry practice, typically relying on RSA or ECDSA signature verification of a bootloader or firmware image against an immutable root-of-trust key. Elliptic Curve Cryptography has been widely adopted for this purpose because it offers equivalent security to RSA at substantially smaller key sizes, which is advantageous for constrained devices with limited flash and RAM. Prior work in this space has extensively documented the use of ECDH for key agreement, following the framework standardized by NIST SP 800-56A, and ECDSA for digital signatures, as specified in FIPS 186-5, over standard curves such as NIST P-256.

For payload confidentiality, AES in Galois/Counter Mode (AES-256-GCM), standardized in FIPS 197 and NIST SP 800-38D, has been the dominant choice for authenticated encryption in both general-purpose and embedded systems, owing to its wide hardware support and extensive cryptanalytic scrutiny. However, AES-GCM's performance on devices lacking dedicated AES instructions or coprocessors is comparatively poor, and its counter-mode construction requires careful nonce management to avoid catastrophic security failures under nonce reuse. These limitations motivated NIST's multi-year Lightweight Cryptography (LWC) standardization effort, which concluded with the selection of the ASCON family of permutation-based authenticated encryption and hashing algorithms as the basis for a new lightweight cryptography standard for constrained devices.

Despite this body of work, relatively few published embedded-systems case studies combine an ECC-based secure boot and key-establishment layer with an ASCON-based payload cipher in a single, end-to-end pipeline validated on real embedded hardware. Existing secure boot literature tends to treat firmware verification in isolation from session-level communication security, while lightweight-cryptography literature tends to evaluate ASCON's cryptographic and performance properties in isolation from a complete secure-boot-to-secure-communication pipeline. This gap motivates the present work: an integrated pipeline in which ECC-based secure boot, key exchange, and mutual authentication establish a trusted communication channel that is then protected using ASCON-128a rather than AES-256-GCM, together with an implementation that makes explicit which stages have been realized on which hardware platform.

3. PROPOSED SYSTEM ARCHITECTURE

The proposed architecture organizes device security into two cooperating layers: a boot-time integrity layer and a session-level communication security layer. The boot-time layer verifies, before any application code executes, that the firmware image currently loaded on the device matches a signature produced by an authorized issuer, using an ECDSA signature verified against a provisioned root-of-trust public key. Only after this check succeeds does the device proceed to establish a secure communication session with a peer node.

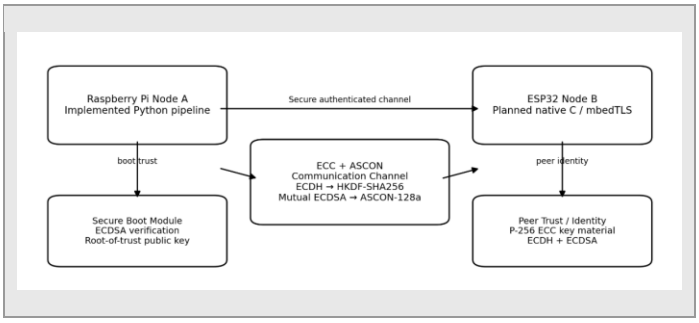


Fig. 1. High-level system architecture showing the Raspberry Pi and ESP32 nodes, the secure boot module, and the ECC/ASCON communication channel between devices.

The session-level layer is itself composed of four sequential stages, illustrated in Fig. 2: an ECDH key exchange over the P-256 curve to establish a shared secret between two communicating nodes; an HKDF-SHA256 key-derivation step that expands the ECDH shared secret into a session key suitable for use with the AEAD cipher; a mutual ECDSA challenge-response step in which each node proves possession of its private key to the other before any application payload is exchanged; and, finally, ASCON-128a authenticated encryption of the application payload, which in the present demonstration is a representative patient health record. The pipeline concludes with a defined secure-shutdown procedure that discards session key material from memory.

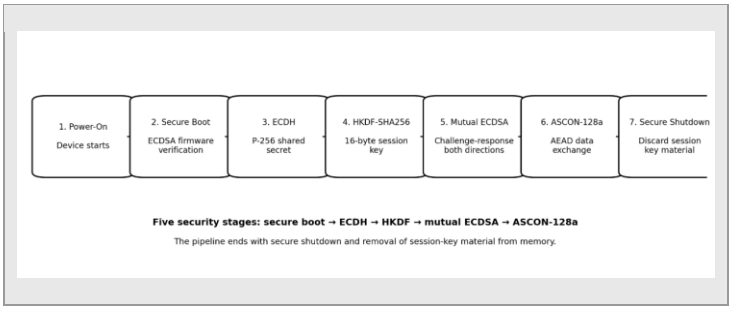


Fig. 2. Complete end-to-end security pipeline from device power-on through secure boot, ECDH key exchange, HKDF key derivation, mutual ECDSA authentication, ASCON-128a data exchange, and secure shutdown.

A. Hardware Components

TABLE I

Component	Role
Raspberry Pi	Primary compute node; hosts the implemented Python security pipeline
ESP32	Target constrained node for planned native C/mbedTLS ASCON-128a port
P-256 ECC keypair	Device identity for ECDH and ECDSA operations
Root-of-trust key	Reference public key used for secure boot signature verification

Hardware components and their roles in the current and planned deployment.

B. Software Stack

TABLE II

Layer	Technology
Language / runtime	Python 3 (Raspberry Pi implementation)
ECC primitives	P-256 ECDH, ECDSA (via a standard Python cryptography library)
Key derivation	HKDF-SHA256
AEAD cipher	ASCON-128a (128-bit key, 128-bit nonce, 128-bit tag)

Software stack used in the Raspberry Pi implementation.

This layered design deliberately separates the asymmetric, higher-cost operations (ECDSA verification, ECDH, HKDF) — which occur once per boot cycle or once per session — from the symmetric, lower-cost ASCON-128a operations, which are executed for every application message. This separation allows the comparatively expensive elliptic-curve operations to be amortized over a session while keeping the per-message cryptographic overhead of ASCON-128a minimal, which is particularly important for constrained targets such as the ESP32 in the planned future deployment.

4. CRYPTOGRAPHIC PIPELINE AND ALGORITHMS

A. Secure Boot Verification

At power-on, the device computes a cryptographic digest of the current firmware image and verifies an accompanying ECDSA signature over that digest using the provisioned root-of-trust public key. Verification succeeds only if the signature is valid for the exact firmware digest presented; any modification to the firmware image, however small, changes the digest and causes verification to fail.

$$Verify(Q, digest, signature) \rightarrow \{ACCEPT, REJECT\} \quad (1)$$

where Q denotes the root-of-trust ECDSA public key. Firmware images that fail verification are rejected and the boot process is halted before any application code is executed.

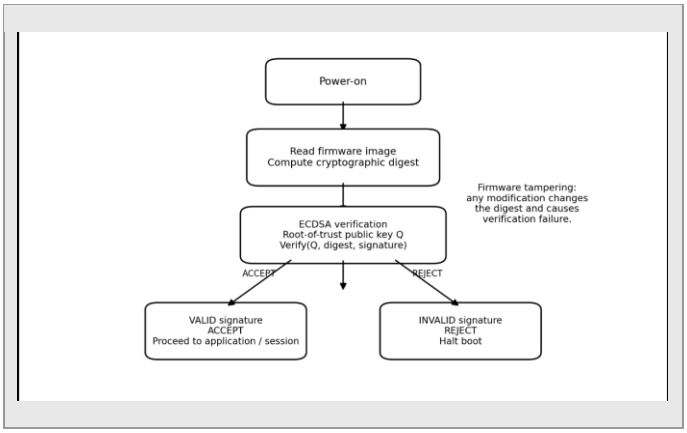


Fig. 3. Secure boot and firmware-tampering detection flow, showing signature verification against the root-of-trust key and rejection of a modified firmware image.

B. ECDH Key Exchange

Following a successful secure boot, the two communicating nodes establish a shared secret using Elliptic Curve Diffie-Hellman over the NIST P-256 curve. Each node generates an ephemeral or static key pair (d, Q = d·G), exchanges public keys with its peer, and computes the shared secret as a scalar multiplication of its own private key with the peer's public key:

$$Z = d_A \cdot Q_B = d_B \cdot Q_A \quad (2)$$

Because scalar multiplication over an elliptic curve group is commutative in this sense, both parties independently arrive at the same shared point Z, the x-coordinate of which is used as the raw shared secret in the subsequent key-derivation stage.

C. HKDF-SHA256 Key Derivation

The raw ECDH shared secret is not used directly as a symmetric key. Instead, it is passed through the HMAC-based Key Derivation Function (HKDF) using SHA-256, which first extracts a pseudorandom key from the shared secret and then expands it into session key material of the length required by ASCON-128a:

$$PRK = HMAC\text{-}Extract(salt, Z) \quad (3)$$

$$K_session = HMAC\text{-}Expand(PRK, info, L) \quad (4)$$

where L is the required output length (16 bytes for the ASCON-128a key) and info is an application-specific context string that binds the derived key to this particular session.

D. Mutual ECDSA Authentication

Before any application payload is exchanged, each node authenticates the other through a challenge-response exchange: node A sends a random challenge to node B, which signs it using its private ECDSA key; node A verifies the signature using node B's known public key, and the process is repeated in the opposite direction so that both nodes mutually confirm each other's identity prior to data exchange.

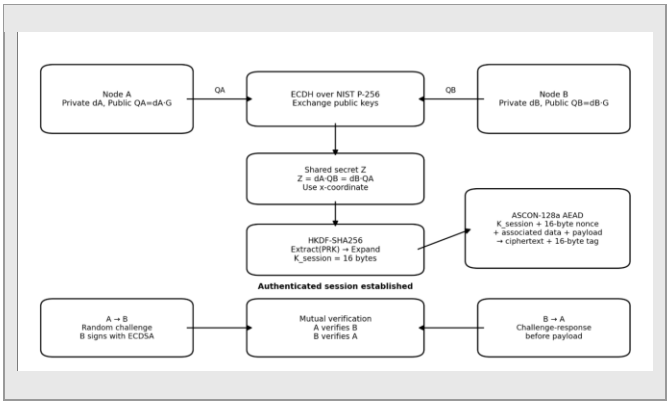


Fig. 4. Detailed cryptographic workflow: ECDH shared-secret computation, HKDF-SHA256 expansion, mutual ECDSA challenge-response, and ASCON-128a encryption/decryption with associated data.

E. ASCON-128a Authenticated Encryption

Application payloads are protected using ASCON-128a, a permutation-based authenticated encryption with associated data (AEAD) cipher built on a sponge-like construction. Given the session key K_session, a 16-byte nonce N, plaintext P, and optional associated data A (unencrypted but authenticated metadata), encryption produces a ciphertext C and a 16-byte authentication tag T:

$$(C, T) = ASCON\text{-}128a.Enc(K_session, N, A, P) \quad (5)$$

$$P' = ASCON\text{-}128a.Dec(K_session, N, A, C, T) \quad (6)$$

Decryption in Eq. (6) returns the recovered plaintext P' only if the supplied tag T verifies correctly against the ciphertext, nonce, and associated data; if any of these inputs has been altered, decryption fails and no plaintext is released, providing both confidentiality and integrity/authenticity for the payload. In the implemented system, the key is 16 bytes, the nonce is 16 bytes, and the tag is 16 bytes, consistent with the ASCON-128a parameter set; associated data support is used to bind non-secret metadata, such as a record identifier, to the encrypted payload without exposing that metadata to encryption.

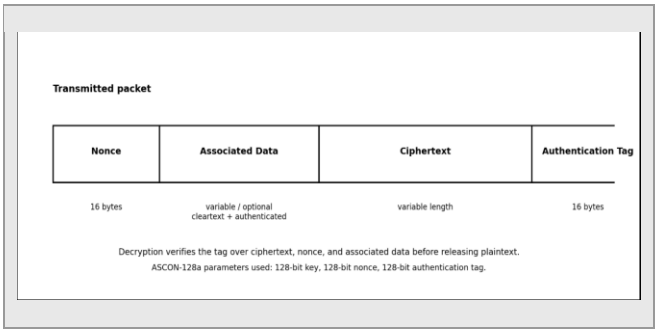


Fig. 5. Structure of an ASCON-128a encrypted packet, showing the 16-byte nonce, ciphertext, associated data, and 16-byte authentication tag fields.

Fig. 5 illustrates the resulting packet structure used in the implementation: a cleartext nonce field, an optional cleartext associated-data field, the ciphertext, and the trailing authentication tag, allowing a receiver to reconstruct the inputs required for Eq. (6).

5. RASPBERRY PI IMPLEMENTATION

The full five-stage pipeline described in Section IV has been implemented in Python on a Raspberry Pi platform. Standard, well-reviewed cryptographic libraries are used for the elliptic-curve and hash-based primitives (ECDH, ECDSA, HKDF-SHA256), while the ASCON-128a AEAD cipher is implemented following its published specification. The implementation is organized as a sequence of discrete, testable stages corresponding to Eqs. (1)–(6): boot-time signature verification, key exchange, key derivation, mutual authentication, and payload encryption/decryption, with an explicit secure-shutdown routine that clears session key material from process memory at the end of a session.

As a representative application payload, the implementation demonstrates encryption, decryption, and tamper detection for a synthetic patient health record, chosen to reflect a realistic IoT use case (e.g., a wearable or bedside medical sensor transmitting readings to a gateway device). The record is passed through the ASCON-128a AEAD stage as described in Section IV-E, with a record identifier carried as associated data.

TABLE III

Pipeline Stage	Raspberry Pi	ESP32
Secure boot (ECDSA verify)	Implemented, validated	Planned
ECDH key exchange	Implemented, validated	Planned
HKDF-SHA256	Implemented, validated	Planned
Mutual ECDSA auth.	Implemented, validated	Planned
ASCON-128a AEAD	Implemented, validated	Planned (native C / mbedTLS)

Implementation status of each pipeline stage, by target platform.

Table III makes explicit the current boundary of the implementation: every pipeline stage has been implemented and functionally validated on the Raspberry Pi. Porting the ASCON-128a stage (and, more generally, the full pipeline) to the ESP32 microcontroller is planned future work requiring a native C implementation, most likely leveraging the mbedTLS library's cryptographic primitives together with a dedicated ASCON-128a implementation, since ASCON is not yet part of mbedTLS's default primitive set. No ESP32 implementation or ESP32-side results are reported in this paper.

6. EXPERIMENTAL RESULTS AND SECURITY ANALYSIS

A. Functional Validation

The implemented pipeline was exercised through a series of functional tests covering both correct-operation and adversarial scenarios. Table IV summarizes the outcomes. In the correct-

operation cases, secure boot verification accepted a validly signed firmware image, the ECDH shared secrets computed independently by the two communicating nodes matched, the HKDF-derived session keys matched, mutual ECDSA authentication succeeded in both directions, and ASCON-128a correctly encrypted and subsequently decrypted the representative patient-record payload, recovering the original plaintext.

TABLE IV

Test Case	Expected Outcome	Observed Result
Secure boot verification (valid firmware)	ACCEPT	PASS
Secure boot verification (tampered firmware)	REJECT	PASS
ECDH shared secret (A vs. B)	MATCH	PASS
HKDF-derived session key (A vs. B)	MATCH	PASS
Mutual ECDSA challenge-response	ACCEPT (both directions)	PASS
ASCON-128a encryption/decryption	Plaintext recovered	PASS
Ciphertext tamper detection	Decryption fails	PASS
One-bit ciphertext modification	Rejected	PASS

Functional and adversarial test results for the implemented Raspberry Pi pipeline.

In the adversarial cases, a deliberately tampered firmware image was correctly rejected at the secure boot stage, an intercepted-and-modified ASCON-128a ciphertext was correctly rejected during decryption, and a single-bit modification introduced into an otherwise valid ciphertext was likewise detected and rejected, confirming that ASCON-128a's authentication tag provides the intended integrity guarantee against even minimal ciphertext alterations.

B. Timing Characteristics

Table V reports mean execution times for the asymmetric-cryptography stages of the pipeline (key generation, ECDH, HKDF, and ECDSA sign/verify), measured on the Raspberry Pi. These stages are executed once per boot cycle or once per session and are independent of the choice of symmetric payload cipher, so the reported figures apply equally to the AES-256-GCM and ASCON-128a variants of the design.

TABLE V

Operation	Mean Time (ms)
ECC key generation	8.2
ECDH shared-secret computation	12.4
HKDF-SHA256 derivation	0.8
ECDSA signature generation	9.8
ECDSA signature verification	7.9
Secure boot verification (end-to-end)	125

Mean execution time of asymmetric pipeline stages on the Raspberry Pi (session/boot-time operations).

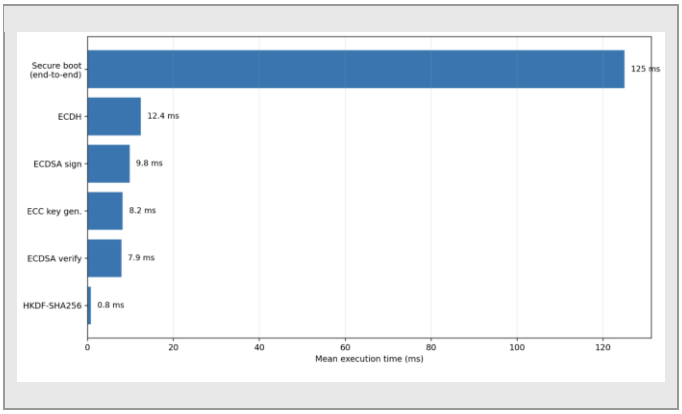


Fig. 6. Comparative performance overview of the asymmetric pipeline stages (secure boot, ECDH, HKDF, ECDSA sign/verify) measured on the Raspberry Pi.

Dedicated timing benchmarks for the ASCON-128a encryption and decryption operations themselves have not yet been collected; the results reported in Table IV establish that ASCON-128a functions correctly on the Raspberry Pi, but per-message throughput and latency figures for ASCON-128a are left for future measurement and are not claimed here. Reporting an unmeasured value would misrepresent the current state of validation.

C. Security Discussion

The combination of ECDSA-based secure boot with mutual ECDH/ECDSA-based session establishment provides defense against two related threats: unauthorized firmware modification, which is caught at boot time before any network activity occurs, and impersonation of a communicating peer, which is caught during mutual authentication before any application data is exchanged. Layering ASCON-128a AEAD on top of this authenticated channel then protects the confidentiality and integrity of the application payload itself, with the observed tamper-detection results (Table IV) indicating that the implementation correctly rejects both wholesale ciphertext substitution and minimal single-bit modification, consistent with the expected behavior of a properly implemented AEAD scheme.

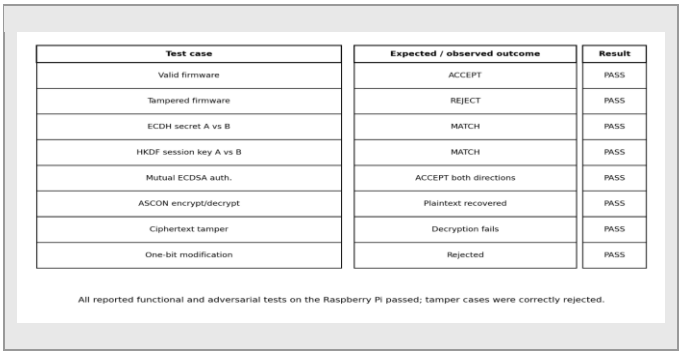


Fig. 7. Summary of functional security test outcomes across the pipeline, showing PASS results for correct-operation cases and correct REJECT/MISMATCH outcomes for adversarial tamper cases.

D. AES-256-GCM vs. ASCON-128a

Table VI summarizes the qualitative differences between the project's original AES-256-GCM design and the ASCON-128a design implemented and validated in this work.

TABLE VI

Property	AES-256-GCM (Original Design)	ASCON-128a (Implemented)
Construction	Block cipher + Galois counter mode	Permutation-based sponge AEAD
Key size	256 bits	128 bits
Nonce size	96 bits (typical)	128 bits
Tag size	128 bits	128 bits
Hardware dependence	Benefits greatly from AES-NI/AES coprocessor	Efficient without dedicated hardware acceleration
Suitability for MCUs	Heavier on devices without AES hardware	Designed for lightweight/constrained targets
Standardization	FIPS 197 / NIST SP 800-38D	NIST lightweight cryptography standard (Ascon family)
Status in this project	Superseded design; not implemented in current codebase	Implemented and validated on Raspberry Pi

Qualitative comparison of the originally specified AES-256-GCM payload cipher and the implemented ASCON-128a payload cipher.

This comparison is qualitative rather than an empirical head-to-head benchmark: no AES-256-GCM implementation was retained or re-benchmarked on the project's hardware for direct comparison, since the current codebase implements ASCON-128a only. The comparison instead reflects documented structural and standardization differences between the two ciphers relevant to the constrained-device use case targeted by this project.

7. LIMITATIONS AND FUTURE WORK

Several limitations of the current implementation should be noted explicitly. First, the ASCON-128a stage, and indeed the entire five-stage pipeline, has been implemented and validated only on the Raspberry Pi platform, using a Python implementation; no functional or performance results are reported for the ESP32 microcontroller, and no such implementation currently exists in the project codebase. Second, dedicated performance benchmarking of ASCON-128a's throughput and latency has not yet been carried out; the timing results reported in this paper (Table V) cover only the asymmetric-cryptography stages of the pipeline, which are independent of the payload cipher choice. Third, the security analysis presented here is limited to functional and tamper-detection testing; it does not include formal cryptographic proofs, side-channel analysis, or third-party security audit, all of which would be necessary before deployment in a production setting.

The primary direction for future work is a native C implementation of the ASCON-128a AEAD stage — and ultimately the full pipeline — for the ESP32 microcontroller, leveraging the mbedTLS library for the underlying ECC and hash primitives together with a dedicated ASCON-128a implementation, since ASCON is not currently included among mbedTLS's built-in algorithms. This future implementation would allow direct performance comparison between the Raspberry Pi and ESP32 deployments and would enable end-to-end testing of the pipeline across genuinely heterogeneous, resource-constrained IoT nodes. Additional future work includes dedicated ASCON-128a throughput/latency benchmarking on both platforms, integration of secure key storage (e.g., a hardware secure element) for the root-of-trust and long-term ECC keys, and formal or third-party security evaluation of the complete implementation.

8. CONCLUSION

This paper presented a layered secure boot and authenticated-communication pipeline for embedded IoT devices, combining ECC-based secure boot verification, ECDH key exchange, HKDF-SHA256 key derivation, and mutual ECDSA authentication with ASCON-128a authenticated encryption for payload confidentiality and integrity, in place of the project's originally specified AES-256-GCM cipher. The complete pipeline was implemented in Python and functionally validated on a Raspberry Pi platform, with test results confirming correct operation across all pipeline stages and correct rejection behavior under adversarial tampering, including detection of a single-bit ciphertext modification. Porting the ASCON-128a stage to the ESP32 microcontroller via a native C/mbedTLS implementation remains future work. The results support the use of ASCON-128a as a practical, lightweight alternative to AES-256-GCM for authenticated communication in resource-constrained embedded IoT systems.

9. REFERENCES

[1] IEEE Transactions on Computers, "Secure boot and firmware integrity verification for embedded systems," IEEE, various issues.

[2] National Institute of Standards and Technology, "Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography," NIST Special Publication 800-56A Rev. 3, 2018.

[3] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," FIPS PUB 197, 2001.

[4] National Institute of Standards and Technology, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, 2007.

[5] National Institute of Standards and Technology, "Digital Signature Standard (DSS)," FIPS PUB 186-5, 2023.

[6] National Institute of Standards and Technology, "Ascon-Based Lightweight Cryptography Standards for Constrained Devices," NIST Special Publication 800-232, Aug. 2025.